

Webprogrammierung: DOM-Manipulation & Modernes Event- Handling

Fachlehrer Stefan Sauer
THWS Geovisualisierung

DOM-Manipulation & Modernes Event-Handling

Übersicht:

- DOM-Baumstruktur & Selektion (`querySelector`)
- Dynamische DOM-Manipulation (`createElement` , `classList`)
- Modernes Event-Handling (`addEventListener` , `event.target`)
- Interaktive Praxis-Beispiele (z. B. Hamburger-Menü)

Die DOM-Baumstruktur & Selektion

Das Document Object Model: Dokument-, Element- und Textknoten

Gezieltes Selektieren: `querySelector()` und `querySelectorAll()`

Traversierung im Baum: `parentElement`, `children`, `closest()`

Entfall: Keine Vorgehensweisen mit `document.write()` oder alten `getElementsByName`-Tricks.

Das Document Object Model (DOM) & Knotenarten

- **Was ist das DOM?**
 - Die objektorientierte Repräsentation des HTML-Dokuments als **Baumstruktur** im Arbeitsspeicher des Browsers.
 - Ermöglicht JavaScript den dynamischen Lese- und Schreibzugriff auf Inhalt, Struktur und CSS-Style
- **Die 3 wichtigsten Knotenarten (*Node Types*):**
 1. **Dokument-Knoten (`document`)**: Die Wurzel des gesamten Baums.
 2. **Element-Knoten (`Element`)**: HTML-Tags (z. B. `<h1>` , `<div>` , `<button>`).
 3. **Text-Knoten (`Text`)**: Der tatsächliche Textinhalt innerhalb von Elementen.

Analogie:

**HTML ist der statische Bauplan;
das DOM ist das lebendige Bauwerk
im Arbeitsspeicher!**

Gezieltes Selektieren: `querySelector` & `querySelectorAll`

Der moderne Standard zur Elementauswahl mittels flexibler **CSS-Selektoren**:

- `document.querySelector(selector)` :
→ Liefert das **erste passende Element** zurück (oder `null`).

```
const titel = document.querySelector("h1");           // Per Tag-Name
const mainBtn = document.querySelector("#submitBtn"); // Per ID (#)
const activeCard = document.querySelector(".card.active"); // Per Klasse (.)
```

- `document.querySelectorAll(selector)` :
→ Liefert eine **NodeList** (Sammlung) **aller passenden Elemente** zurück.

```
const alleButtons = document.querySelectorAll("button.btn");
alleButtons.forEach(btn => btn.disabled = true);
```

Traversierung im DOM-Baum (Navigieren)

Gezieltes Wandern zwischen Eltern-, Geschwister- und Kind-Elementen:

- **parentElement** : Springt zum übergeordneten Vater-Element.
- **children** : Liefert alle direkten Kind-Elemente als Sammlung.
- **closest(selector)** : Wandert nach **oben** und sucht das nächste passende Eltern-Element (ideal für Event-Handling!).

```
const item = document.querySelector(".list-item");

const elternNode = item.parentElement; // Vater-Element
const form = item.closest("form");    // Nächstes <form> nach oben
const kinder = form.children;         // Direkte Kinder von form
```


Übersicht: DOM-Selektion & Traversierung

Methode / Eigenschaft	Funktion	Rückgabewert
<code>querySelector('sel')</code>	Erstes Treffer-Element	Element oder <code>null</code>
<code>querySelectorAll('sel')</code>	Alle Treffer-Elemente	<code>NodeList</code>
<code>parentElement</code>	Übergeordnetes Vater-Element	Element
<code>children</code>	Direkte Kind-Elemente	<code>HTMLCollection</code>
<code>closest('sel')</code>	Nächstes Eltern-Element nach oben	Element oder <code>null</code>

Best Practice:

- Nutze ausschließlich `querySelector` / `querySelectorAll` mit CSS-Selektoren.
- Veraltete Methoden wie `document.write()` entfallen komplett.

Praxis-Check: DOM-Selektion & Traversierung im Einsatz

- **Ausführbare Demodatei:** `Samples/JS/dom-selection-demo.html`
- **Gezeigte Kernkonzepte im Code:**
 1. `querySelector()` : Greift gezielt das erste Element per ID (`#mainContainer`) oder CSS-Klasse (`.card-item.active`).
 2. `querySelectorAll()` : Selektiert alle Treffer als `NodeList` und durchläuft diese per `.forEach()` .
 3. `parentElement` & `children` : Navigiert eine Ebene nach oben zum Vaterknoten bzw. liest direkte Kindknoten aus.
 4. `closest(selector)` : Wandert im Baum nach oben, um das nächstgelegene passende Eltern-Element zu finden.

Ausprobieren:

Öffne `Samples/JS/dom-selection-demo.html` im Browser und teste die interaktiven Buttons!

Dynamische DOM-Manipulation

(`createElement` & `append`)

Knoten im Speicher erzeugen (`document.createElement`):

```
const newCard = document.createElement("div");  
newCard.textContent = "Neue Karte";
```

In den DOM-Baum einfügen (`append` vs. `prepend`):

- `parent.append(child)` : Fügt das Element am **Ende** der Kinder ein.
- `parent.prepend(child)` : Fügt das Element am **Anfang** der Kinder ein.

HTML-Strings direkt einfügen (`insertAdjacentHTML`):

```
const list = document.querySelector("#kundenListe");  
// Positionen: 'beforebegin', 'afterbegin', 'beforeend', 'afterend'  
list.insertAdjacentHTML("beforeend", `<li class="item">Neuer Kunde</li>`);
```

Klassen & Stile dynamisch steuern (`classList`)

Verwende `classList` zur sauberen Steuerung von CSS-Klassen (Trennung von JS & Styling!):

```
const btn = document.querySelector(".action-btn");

btn.classList.add("active");      // Klasse hinzufügen
btn.classList.remove("disabled"); // Klasse entfernen
btn.classList.toggle("highlight"); // Umschalten (an/aus)

if (btn.classList.contains("active")) {
  console.log("Button ist aktiv!");
}
```

- **Direkte Inline-Styles (nur bei dynamischen Werten):**

```
element.style.backgroundColor = "#005a9c"; // CamelCase-Schreibweise!
```

Data-Attribute nutzen (`data-*` & `dataset`)

Koppelung von HTML-Struktur und JavaScript-Daten über benutzerdefinierte Attribute:

- **HTML-Quellcode:**

```
<button class="user-btn" data-user-id="42" data-role="admin">Profil</button>
```

- **JavaScript-Zugriff über `element.dataset` :**

```
const btn = document.querySelector(".user-btn");

// Auslesen (data-user-id wird automatisch zu CamelCase userId):
console.log(btn.dataset.userId); // "42" (immer vom Typ String!)
console.log(btn.dataset.role);   // "admin"

// Schreiben / Ändern:
btn.dataset.role = "superadmin";
```

Übersicht: Dynamische DOM-Manipulation

Aufgabe	Modernes Verfahren	Nutzen
Element erzeugen	<code>document.createElement('div')</code>	Sicheres Erzeugen von Knoten im Speicher
Element anhängen	<code>parent.append(child)</code>	Fügt Knoten am Ende der Kinder ein
HTML einfügen	<code>e1.insertAdjacentHTML('beforeend', html)</code>	Schnelles Einfügen von Template-Strings

Aufgabe	Modernes Verfahren	Nutzen
Klassen umschalten	<code>el.classList.toggle('active')</code>	Sauberes Trennen von JS & CSS-Design
Metadaten lesen	<code>el.dataset.userId</code>	Zugriff auf <code>data-user-id</code> im HTML

Best Practice:

Nutze `classList` für visuelle Änderungen anstelle von direkten Inline-Styles in `element.style`.

Praxis-Check: Dynamische DOM-Manipulation im Einsatz

- **Ausführbare Demodatei:** `Samples/JS/dom-manipulation-demo.html`
- **Gezeigte Kernkonzepte im Code:**
 1. `document.createElement()` & `.append()` : Erzeugen von Knoten im Speicher und Einfügen am Ende des Ziel-Containers.
 2. `insertAdjacentHTML('beforeend', html)` : Direktes Parsen und Einfügen von HTML-Template-Strings
 3. `classList.toggle('highlight')` : Dynamisches Umschalten von CSS-Klassen für visuellen Status.
 4. `dataset` : Auslesen (`dataset.cardId`) und dynamisches Setzen (`dataset.status`) von Data-Attributen

Ausprobieren:

Öffne `Samples/JS/dom-manipulation-demo.html` im Browser und teste die interaktiven Buttons!

Modernes Event Handling

Interaktivität & Event-Listener

Übersicht:

- **Paradigmenwechsel:** Saubere Trennung von HTML (Struktur) und JS (Verhalten)
- `addEventListener()` : Events wie Klicks oder Eingaben sauber abfangen
- **Das Event-Objekt (`event`):** `event.target` und `event.preventDefault()`
- **Praxis:** Interaktive Steuerung von Webseiten-Elementen

Entfall: Keine Inline-Eventhandler (`onclick=""` im HTML)

Veralteter Ansatz (Anti-Pattern):

```
<!-- NICHT VERWENDEN: Vermischt Struktur mit Logik -->  
<button onclick="speichernData()">Speichern</button>
```

Warum Inline-Handler problematisch sind:

- **Verletzung der Trennung von Belangen (Separation of Concerns):** HTML steuert Struktur/Inhalt, JS das Verhalten.
- **Verschmutzung des globalen Scope:** Funktionen müssen global deklariert sein.
- **Wartbarkeit & Flexibilität:** Schwer zu debuggen und nur ein Handler pro Element möglich.

Modernes Verfahren:

```
<button id="saveBtn">Speichern</button>
```

```
const saveBtn = document.querySelector('#saveBtn');  
  
// Saubere Registrierung des Event-Listeners in JavaScript:  
saveBtn.addEventListener('click', speichernData);
```

- Trennt HTML-Markup und JavaScript-Logik vollständig.
- Erlaubt beliebig viele Listener auf demselben Element.

Event-Listener in der Praxis (addEventListener)

```
const btn = document.querySelector('.action-btn');

// Variante 1: Direkt mit anonymer Pfeilfunktion (Arrow Function)
btn.addEventListener('click', (event) => {
  console.log('Button wurde geklickt!');
});

// Variante 2: Mit benannter Funktion (erhöht Lesbarkeit & Wiederverwendbarkeit)
function handleAction(event) {
  console.log('Aktion ausgeführt!');
}
btn.addEventListener('click', handleAction);
```

Das Event-Objekt (`event`)

Beim Auslösen eines Events übergibt der Browser automatisch ein **Event-Objekt** mit nützlichen Metadaten:

- `event.target` : Das konkrete HTML-Element, das das Ereignis ausgelöst hat.
- `event.preventDefault()` : Unterdrückt das standardmäßige Verhalten des Browsers (z. B. Seiten-Reload beim Formular-Absenden oder Springen bei Links).

```
const link = document.querySelector('#specialLink');

link.addEventListener('click', (event) => {
  event.preventDefault(); // Verhindert das Standard-Springen der Seite
  console.log('Geklicktes Element:', event.target);
});
```

Übersicht: Modernes Event Handling

Konzept	Syntax / Methode	Zweck / Praxis - Nutzen
Listener registrieren	<code>e1.addEventListener('click', fn)</code>	Reagiert auf Benutzerinteraktionen ohne Inline-HTML
Standardverhalten stoppen	<code>event.preventDefault()</code>	Verhindert ungewolltes Neuladen bei Formularen & Links
Auslöser ermitteln	<code>event.target</code>	Referenz auf das konkret angeklickte Element

Praxis-Check: Modernes Event Handling im Einsatz

- **Ausführbare Demodatei:** `Samples/JS/event-handling-demo.html`
- **Gezeigte Kernkonzepte im Code:**
 1. `addEventListener('click', handler)` : Ereignisse sauber im JS-Code abfangen.
 2. `event.preventDefault()` : Verhindern von Standard-Browseraktionen.
 3. `event.target` : Auslesen von Attributen und Werten des geklickten Elements.

Ausprobieren:

Öffne `Samples/JS/event-handling-demo.html` im Browser und teste die interaktiven Buttons und Event-Handler!

Asynchrone Programmierung & Fetch API

Geodaten & externe APIs in JavaScript laden

Übersicht:

- **Synchron vs. Asynchron:** Warum Webseiten beim Datenladen nicht einfrieren dürfen
- **Daten abrufen mit `fetch()`** : Der moderne Browser-Standard für externe Ressourcen
- **Elegante Steuerung mit `async` / `await`** : Asynchroner Code, so einfach lesbar wie synchroner Code
- **Geodaten-Praxis:** GeoJSON-Dateien laden, parsen & im DOM verarbeiten

Warum Asynchronität?

- **Synchroner Ablauf (Problem):**
 - Befehle werden streng nacheinander ausgeführt.
 - Beim Laden großer Dateien (z. B. 5 MB GeoJSON) **blockiert** der gesamte Browser: Die Webseite friert ein, Buttons reagieren nicht mehr!
- **Asynchroner Ablauf (Lösung):**
 - Der Browser fordert die Datei im Hintergrund an.
 - Die Webseite bleibt sofort bedienbar und flüssig.
 - Sobald die Daten eingetroffen sind, verarbeitet eine Funktion das Ergebnis.

Externe Geodaten, Karten-Tiles oder APIs werden in JavaScript immer asynchron geladen!

Daten abrufen mit der `fetch()`-API

Die `fetch()`-API ist die moderne Schnittstelle zur Kommunikation mit Webservern:

```
// Sendet eine HTTP-Anfrage an den Server  
fetch('orte.geojson');
```

- **Entfall veralteter Methoden:**
 - Kein `XMLHttpRequest` (XHR) und kein `jQuery.ajax()` mehr!
- **Das Grundprinzip in 2 Schritten:**
 1. **Anfrage senden:** Server gibt ein Antwort-Objekt (*Response*) zurück.
 2. **JSON parsen:** `response.json()` konvertiert den Text-String in echte JS-Objekte.

Lesbarer Code durch `async` / `await`

Das Schlüsselwort-Paar `async` / `await` macht asynchronen Code extrem übersichtlich:

- `async` : Kennzeichnet eine Funktion als asynchron (`async function ...`).
- `await` : Pausiert die Ausführung *innerhalb* der Funktion, bis die Antwort vorliegt.

```
// Asynchrone Funktion zum Laden von Geodaten
async function ladeGeodaten() {
  // 1. Auf Antwort des Servers warten:
  const response = await fetch('orte.geojson');
  // 2. Auf das Konvertieren der GeoJSON-Daten warten:
  const geodaten = await response.json();

  console.log('Geladene Orte:', geodaten);
}
ladeGeodaten();
```

Fehlerbehandlung mit `try ... catch`

Was passiert, wenn die GeoJSON-Datei nicht existiert (404) oder das Netzwerk ausfällt?

Verwende `try ... catch`, um Fehler abzufangen, ohne dass die Webanwendung abstürzt:

```
async function ladeGeodatenSicher(url) {  
  try {  
    const response = await fetch(url);  
  
    // Prüfen, ob der HTTP-Status erfolgreich war (z. B. Code 200)  
    if (!response.ok) {  
      throw new Error(`HTTP-Fehler! Status: ${response.status}`);  
    }  
    const data = await response.json();  
    return data;  
  } catch (error) {console.error('Fehler beim Geodaten-Abruf:', error.message);}  
}
```

Übersicht: `fetch()` & `async` / `await`

Baustein	Syntax / Befehl	Zweck in der Praxis
<code>async</code>	<code>async function ladeDaten() {..}</code>	Erlaubt die Nutzung von <code>await</code> in der Funktion
<code>fetch()</code>	<code>await fetch('datei.geojson')</code>	Fordert Geodaten/APIs asynchron im Hintergrund an
<code>json()</code>	<code>await response.json()</code>	Konvertiert den GeoJSON - String in ein nutzbares JS - Objekt
<code>try ... catch</code>	<code>try { ... } catch (err) { ... }</code>	Fängt Netzwerkfehler oder falsche Dateipfade ab

Best Practice:

Verwende ausschließlich `async` / `await` in Kombination mit `fetch()` für alle Netzwerkanfragen.

Praxis-Check: Asynchrone Geodaten-Verarbeitung im Einsatz

- **Ausführbare Demodatei:** `Samples/JS/async-fetch-demo.html`
- **Gezeigte Kernkonzepte im Code:**
 1. `async / await` : Sauberer, schrittweiser Ablauf ohne verschachtelte Callbacks.
 2. `fetch('orte.geojson')` : Asynchrones Laden von Geodaten-Dateien.
 3. `try ... catch` & `response.ok` : Zuverlässige Fehlerbehandlung bei Netzwerk- oder Serverprobleme
 4. **DOM-Integration:** Automatische Generierung von UI-Karten aus den geladenen Geodaten.

Ausprobieren:

Öffne `Samples/JS/async-fetch-demo.html` im Browser und teste den asynchronen Geodaten-Abruf!

Clientseitige Speicherung, Web APIs & KI-Workflows

Web Storage, Browser APIs & KI-gestützte Entwicklung (SDD)

- **Die JavaScript Sandbox:** Sicherheitsarchitektur & isolierte Speicherung
- **Clientseitige Speicherung:** `localStorage` vs. `sessionStorage` & JSON-Serialisierung
- **Moderne Web APIs:** Die Geolocation API zur Standortabfrage in GIS-Apps
- **KI-Workflows & SDD:** Spec-Driven Development mit KI-Coding-Assistenten

Clientseitige Speicherung & die JS-Sandbox

JavaScript läuft aus Sicherheitsgründen in einer strikten **Sandbox** im Browser:

- **Sandbox-Prinzip (Same-Origin Policy):**
 - Kein direkter, unkontrollierter Zugriff auf das lokale Dateisystem der Benutzer.
 - Daten sind isoliert pro Domäne/Protokoll gespeichert.
- **Web Storage API (Zwei Speicherarten):**
 - **localStorage** : Speichert Daten **dauerhaft** im Browser-Profil (übersteht Neustarts).
 - **sessionStorage** : Speichert Daten **temporär** (wird beim Schließen des Tabs gelöscht).

```
// Schlüssel-Wert-Paar dauerhaft ablegen  
localStorage.setItem('theme', 'dark');
```

Daten-Serialisierung mit `JSON.stringify` & `JSON.parse`

Web Storage kann ausschließlich **Strings** speichern. Objekte müssen vorher umgewandelt werden!

- **Serialisierung (Objekt → String):** `JSON.stringify(obj)`
- **Deserialisierung (String → Objekt):** `JSON.parse(string)`

```
const userConfig = { layer: 'OSM', zoom: 12 };

// 1. Objekt in JSON-String umwandeln & speichern
localStorage.setItem('userConfig', JSON.stringify(userConfig));

// 2. String auslesen & zurück in JS-Objekt verwandeln
const rawData = localStorage.getItem('userConfig');
const configObj = JSON.parse(rawData);
console.log(configObj.zoom); // 12
```

Praxis-Check: Clientseitige Speicherung im Einsatz

- **Ausführbare Demodatei:** `Samples/JS/storage-demo.html`
- **Gezeigte Kernkonzepte im Code:**
 1. **Sandbox-Isolierung:** Daten bleiben sicher im Web-Browser des Nutzers.
 2. `localStorage` **VS.** `sessionStorage` : Vergleich von permanenter und sitzungsbasierter Speicherung.
 3. `JSON.stringify()` & `JSON.parse()` : Saubere Serialisierung komplexer Einstellungs-Objekte.

Ausprobieren:

Öffne `Samples/JS/storage-demo.html` im Browser und teste das Speichern von Karten-Einstellungen!

Moderne Web APIs: Die Geolocation API

Browser bieten viele eingebaute Schnittstellen (**Web APIs**), die speziell in der Geovisualisierung wichtig sind:

- **navigator.geolocation** : Abfrage des aktuellen Benutzer-Standorts (GPS / IP / WLAN).
- **Datenschutz & Permission Model**: Der Browser verlangt explizit die Zustimmung des Benutzers.

```
// Standort abfragen (asynchron per Callback)
navigator.geolocation.getCurrentPosition(
  (position) => {
    console.log('Breitengrad:', position.coords.latitude);
    console.log('Längengrad:', position.coords.longitude);
  },
  (error) => console.error('Standortfehler:', error.message)
);
```

Praxis-Check: Moderne Web APIs (Geolocation) im Einsatz

- **Ausführbare Demodatei:** `Samples/JS/web-apis-demo.html`
- **Gezeigte Kernkonzepte im Code:**
 1. `'geolocation' in navigator` : Browser-Kompatibilität sicher prüfen.
 2. `getCurrentPosition()` : Asynchroner Aufruf mit Success- und Error-Callbacks.
 3. **Berechtigungsverwaltung:** Sauberes Abfangen von Ablehnungen (`PERMISSION_DENIED`).

Ausprobieren:

Öffne `Samples/JS/web-apis-demo.html` im Browser und teste die Standort-Ermittlung!

KI-gestützte Entwicklung mit Spec-Driven Development (SDD)

Moderne KI-Tools (z. B. Antigravity, Cursor) revolutionieren die Webprogrammierung durch **Spec-Driven Development (SDD)**:

- **Das Prinzip: Spezifikation vor Code:**
 - Statt vager Prompts ("Mach mir eine Karte") wird zuerst eine präzise Spezifikation (`spec.md`) formuliert
 - Die Spezifikation definiert Datenstrukturen (GeoJSON), Schnittstellen, Validierungsregeln und UX-Verhalten.
- **Vorteile für Einsteiger:**
 - KI generiert sauberen, standardkonformen Code nach klaren Vorgaben.
 - Fehler lassen sich durch schrittweises Refactoring und Prompt-Iterationen gezielt beheben.

Übersicht: Speicherung, Web APIs & KI

Thema	Schlüsseltechnologie	Einsatz in der Geovisualisierung
Web Storage	<code>localStorage</code> / <code>sessionStorage</code>	Speichern von Benutzereinstellungen, Filtern & Kartenausschnitten
Web APIs	<code>navigator.geolocation</code>	Abfragen des eigenen Standorts für Routen- & Umgebungskarten
KI & SDD	Spec-Driven Development (SDD)	Spezifikationsgestützte Generierung und Refactoring von JS & GeoJSON

Best Practice:

Nutze `localStorage` für persistenten Status im Browser, binde native Web APIs wie `geolocation` mit sauberem Error-Handling ein und entwickle Anwendungen nach dem SDD-Prinzip auf Basis präziser Spezifikationen.

Formularverarbeitung & Validierung

Benutzereingaben abfangen, Daten extrahieren & native Validierung

Übersicht:

- **Formular-Ereignisse:** `submit`, `input` & `change` richtig einsetzen
- **Datenextraktion mit `FormData`** : Automatische Formularauswertung ohne Handarbeit
- **Native Validierung:** Constraint Validation API (`required`, `checkValidity()`)
- **Praxis-Check:** Interaktives Formular zur Geodaten-Erfassung

Formular-Ereignisse (`submit`, `input`, `change`)

Formulare in JavaScript reagieren auf verschiedene Interaktions-Ereignisse:

- `submit` : Feuert beim Absenden (per Button oder Enter).
→ Wichtig: Stets `event.preventDefault()` aufrufen!
- `input` : Feuert **sofort bei jedem Tastendruck** / Ändern des Textes.
→ Ideal für Live-Validierung, Zeichenzähler oder Echtzeit-Suchfilter.
- `change` : Feuert beim Verlassen des Feldes (`blur`) oder bei Auswahl im Dropdown (`<select>`).

```
// Live-Reaktion auf jeden Tastendruck im Textfeld
inputElem.addEventListener('input', (e) => {
  console.log('Aktueller Text:', e.target.value);
});
```

Die `FormData`-API (Kompakte Datenextraktion)

Statt jedes Formularfeld einzeln per `document.querySelector()` auszulesen, liest `FormData` das gesamte Formular automatisch auf einmal aus:

```
const form = document.querySelector('#poiForm');

form.addEventListener('submit', (event) => {
  event.preventDefault();

  // 1. FormData aus dem Formular-Element erzeugen (erfordert name="..." Attribute)
  const formData = new FormData(form);

  // 2. Direkt in ein einfaches JavaScript-Objekt umwandeln
  const dataObjekt = Object.fromEntries(formData);

  console.log(dataObjekt); // { title: 'Festung', category: 'sightseeing' }
});
```

Native Formularvalidierung (Constraint Validation API)

Browser bieten eingebaute Prüfungen, um fehlerhafte Eingaben vor dem Absenden abzufangen:

- **HTML5-Attribute:** `required`, `minlength="3"`, `type="email"`, `pattern="..."`
- **JavaScript-Validierung:**
 - `form.checkValidity()` : Liefert `true`, wenn alle Felder gültig sind.
 - `input.validity.valid` : Einzelstatus eines Feldes abfragen.
 - `form.reportValidity()` : Stoppt den Ablauf und zeigt die HTML5-Fehlerblasen an.

```
if (!form.checkValidity()) {  
    console.log('Formular ist unvollständig!');  
    form.reportValidity(); // Fehlermeldungen anzeigen  
    return;  
}
```

Übersicht: Formularverarbeitung & Validierung

Event	Syntax / API	Zweck / Praxis - Nutzen
submit	<code>form.addEventListener('submit', fn)</code>	Formular - Absenden abfangen (stets <code>preventDefault()</code>)
input	<code>input.addEventListener('input', fn)</code>	Reagiert sofort bei jedem Tastendruck (Live - Suche)
change	<code>select.addEventListener('change', fn)</code>	Reagiert bei Wertänderung in Dropdowns oder nach Fokusverlust
FormData	<code>new FormData(form)</code>	Liest alle Eingabefelder automatisch als Key - Value aus
Validierung	<code>form.checkValidity()</code>	Prüft native HTML5 - Regeln vor der Weiterverarbeitung

Praxis-Check: Formularverarbeitung & Validierung im Einsatz

- **Ausführbare Demodatei:** `Samples/JS/form-validation-demo.html`
- **Gezeigte Kernkonzepte im Code:**
 1. **input & change Events:** Live-Zeichenzähler und Sofort-Reaktion bei Dropdown-Auswahl.
 2. **FormData API:** Datenextraktion mit `new FormData(form)` & `Object.fromEntries()` .
 3. **Constraint Validation:** Native Validierung mit `checkValidity()` & `validity.typeMismatch` .

Ausprobieren:

Öffne `Samples/JS/form-validation-demo.html` im Browser und teste die Formular-Validierung & Extraktion!

Zurück zum Maik: Ein Hamburger Menü (10)

- **Begriff & Form:** Icon mit drei waagerechten, parallelen Strichen; benannt nach der optischen Ähnlichkeit zu den drei Lagen eines Hamburgers. Symbolisiert eine Menüliste.
- **Einsatzzweck:** Platzsparende Navigation auf kleinen Bildschirmen (Smartphones/Tablets) im Responsive Webdesign.



- **Kritik & Usability:**

- *Vorteil:* Sehr hohe Wiedererkennbarkeit & aufgeräumtes Layout.

- *Nachteil:* Versteckt Navigationsoptionen und erfordert einen zusätzlichen Klick.

- **HTML- & Unicode-Ersetzung:**

- ☰ (`☰` – *TRIGRAM FOR HEAVEN*)

- ☷ (`≡` – *IDENTICAL TO*)

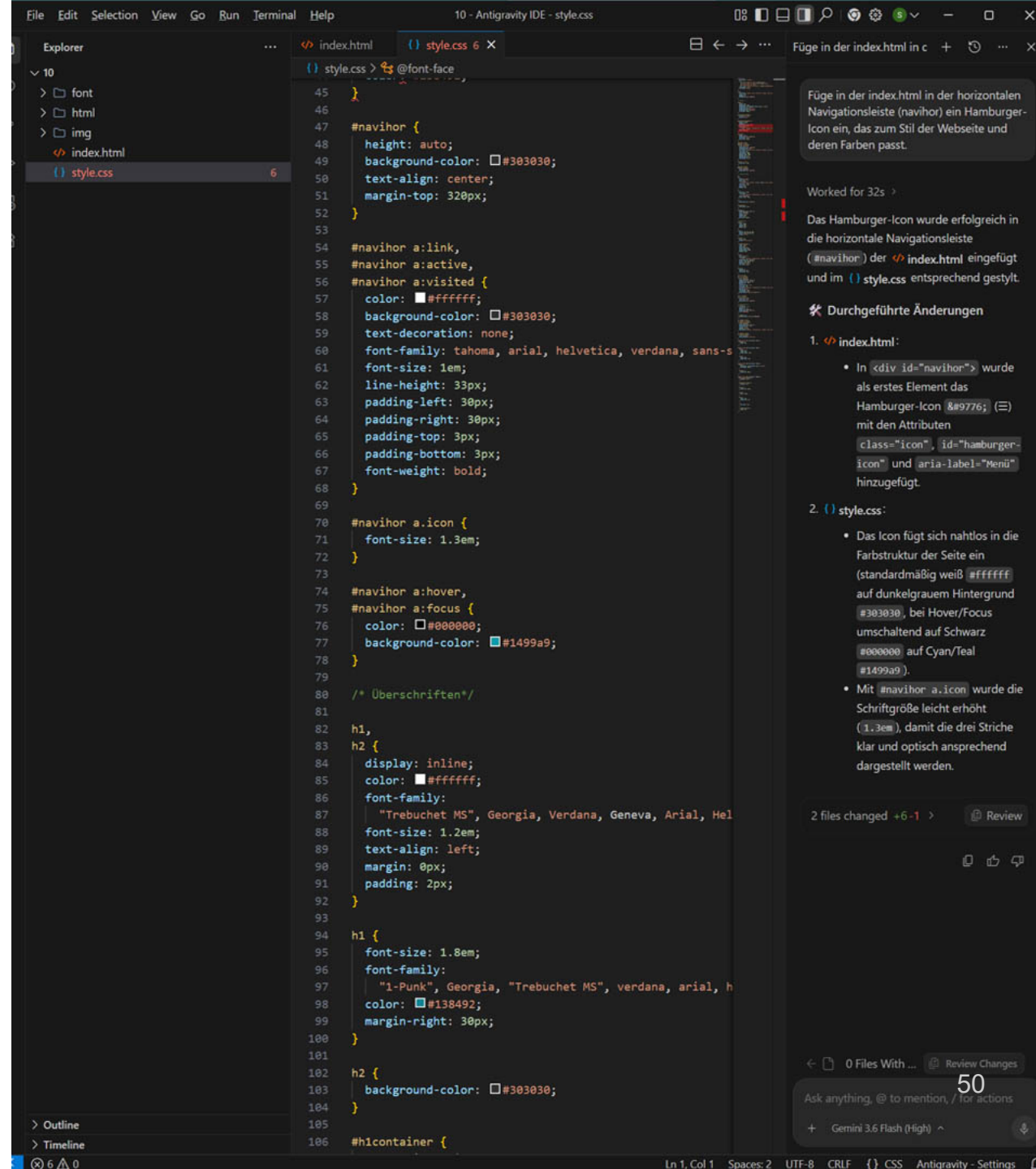
Lösung mit Antigravity

- Kopieren der Webseite in einen neuen Ordner (10)
- Öffnen des Ordners in Antigravity
- Einfügen des Prompts:

"..

Füge in der index.html in der horizontalen Navigationsleiste (navihor) ein Hamburger-Icon ein, das zum Stil der Webseite und deren Farben passt..."

[zurück zur Übersicht](#)



Händische Lösung

index.html

Ergänzen der Navihor:

```
<div id="navihor">  
  <a href="#" class="icon" id="hamburger-icon" aria-label="Menü">&#9776;</a>  
  <a href="#">Kontakt</a>  
  <a href="#">Impressum</a>  
  <a href="#">Datenschutz</a>  
</div>
```

Das Attribut `aria-label="Menü"` stellt zusätzlich die Barrierefreiheit für Screenreader sicher.

style.css

Einfügen des neuen Styles für den Hamburger

```
#navihor a.icon {  
    font-size: 1.3em;  
}  
#navihor a:hover,  
#navihor a:focus {  
    color: #000000;  
    background-color: #1499a9;  
}
```

Der Hamburger sieht gut aus, soll aber erst ab 768 Pixeln abwärts sichtbar sein.

→ Prompt:

"Der Hamburger-Button passt so. Er soll jedoch nur sichtbar sein, wenn das Hauptmenü im aside>nav-Container ab 768 Pixeln über den main-Container rutscht.

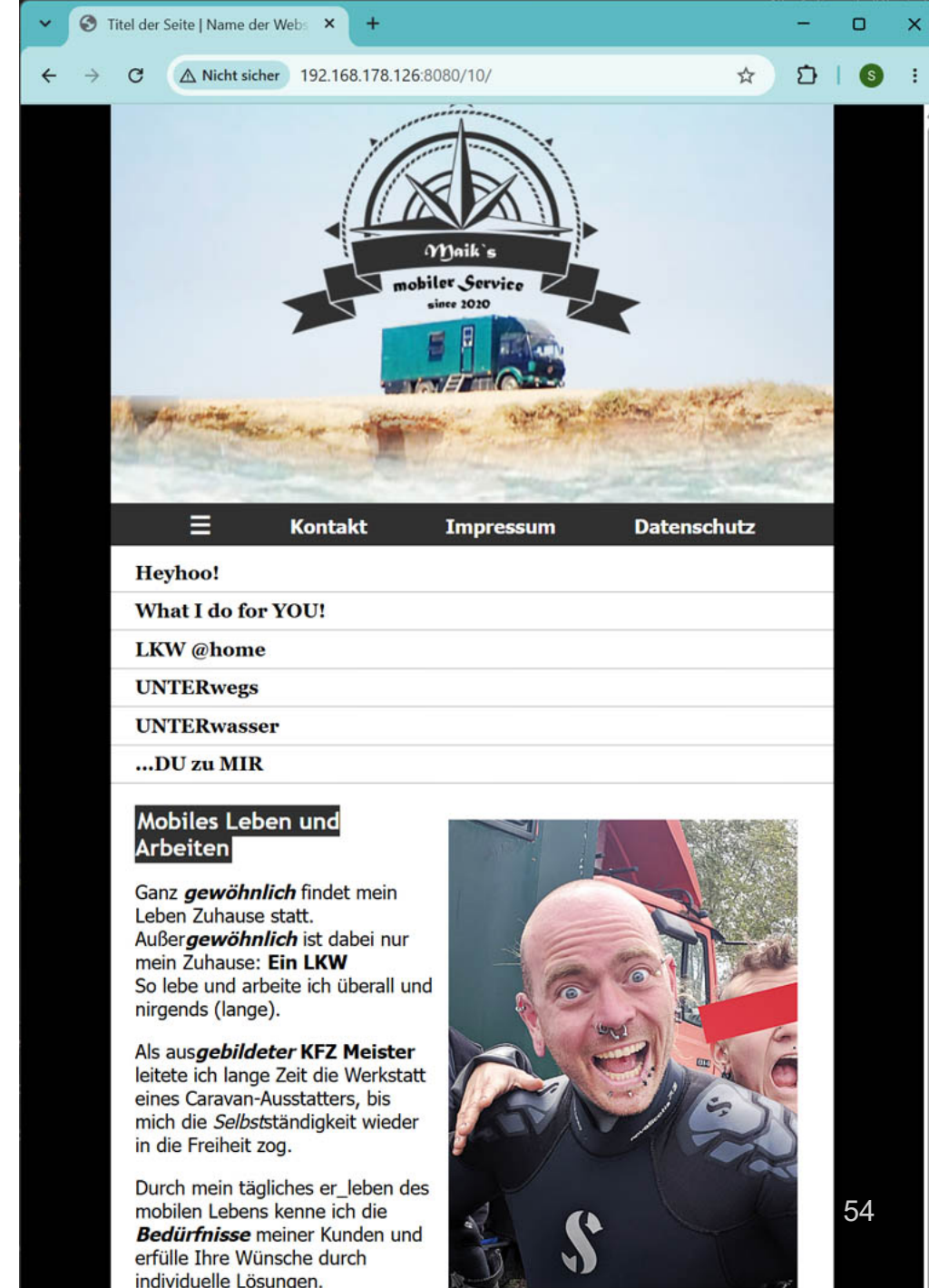
Passe also dementsprechend im css die @Media-Styles an, so dass der Hamburger über 768 Pixeln unsichtbar ist."

```
@media screen and (min-width: 768px) {  
  #navihor a.icon {  
    display: none;  
  }  
  aside { /*alles so belassen, wie es ist*/}  
  main { /*alles so belassen, wie es ist*/}  
}
```

- Webseite mit Hamburger-Button
- Testen der Media-Queries
- Öffnen der "Entwicklertools" (F12)
- Ein/Ausblenden des Handy-Menüs

→ Aktuell ohne Interaktion

[zurück zur Übersicht](#)



JS-Interaktion

Mit dem Hamburger soll sich nun das vertikale Menü öffnen und schließen lassen.

→ Prompt:

"Erstelle nun einen Ordner "js" in dem Du zukünftig alle erzeugten Javascript-Dateien anlegst.

Erstelle nun ein JS mit dem die Hauptnavigation im aside>nav-Container mit einem Klick ein- und ausgeblendet werden kann.

Das JS soll simpel, leicht verständlich und gut kommentiert sein."

Händische Lösung

- Erstellen eines neuen Javascriptes im neuen Unterordner "js" (siehe nächste Seite)
- Einbindung in index.html im Head-Bereich:

```
<script src="js/script.js" defer></script>
```



```
// Warten, bis der DOM-Inhalt der Webseite vollständig geladen ist
document.addEventListener("DOMContentLoaded", function () {
  // 1. Auswählen des Hamburger-Buttons (über seine ID)
  const hamburgerBtn = document.getElementById("hamburger-icon");

  // 2. Auswählen der Hauptnavigation im aside>nav-Container
  const navMenu = document.querySelector("aside nav");

  // Überprüfen, ob beide Elemente auf der Seite vorhanden sind
  if (hamburgerBtn && navMenu) {

    // 3. Event-Listener für das Klick-Ereignis hinzufügen
    hamburgerBtn.addEventListener("click", function (event) {
      // Standard-Linkverhalten des <a>-Tags (Seiten-Springen durch #) verhindern
      event.preventDefault();

      // 4. Klasse 'active' umschalten:
      navMenu.classList.toggle("active");
    });
  }
});
```

Anpassungen im css:

```
@media screen and (max-width: 767px) {  
  aside nav {  
    display: none;  
  }  
  aside nav.active {  
    display: block;  
  }  
}  
  
@media screen and (min-width: 768px) {  
  #navihor a.icon {  
    display: none;  
  }  
  aside nav {  
    display: block;  
  }  
  ...}
```

Was ist nötig, damit die Navigation auch auf der LKW @home Seite funktioniert?

- Einbinden des JS (auf korrekte Pfade achten)
- Anpassen des Headers

→ Prompt:

"Passe nun die html-Seite "mobiles-Leben-im-LKW.html" so an, dass hier der Hamburger identisch eingebunden ist und funktioniert."

Wie gehts weiter? (11)

Für einen fertigen Web-auftritt gibt es noch viel zu tun:

- weitere Unterseiten erstellen
- Funktionen einbinden
- Standort
- Kontaktformular

→ Prompt:

"Erstelle nun eine Unterseite "kontakt.html" im html Ordner im gleichen Stil wie die anderen beiden Unterseiten und binde in diese Datei mehrere Article ein:

Article-Text:

"Kontaktadresse:

Maik Anonym, Maikstrasse 66, Berlin"

Article-Text:

"Mein aktueller, ungefährender Standort"

Füge darunter eine interaktive OpenStreetMap Karte ein, die den Standort Berlin anzeigt.

Article-Text:

"Du suchst Kontakt zu Gleichgesinnten und Selbstausbauerinnen, dann fülle das Formular aus und sende mir eine E-Mail"

Darunter fügst Du ein Formular ein, das folgende Felder enthält:

Vorname (required, text), Nachname (required, text),

Mein Fahrzeug (Dropdown mit Auswahlliste: Sonstiges, PKW, LKW, Wohnmobil, Wohnwagen),

Mein Anliegen (required, Textfeld mit folgendem Standardtext

"...das Reh springt hoch, das Reh springt weit, warum auch nicht, es hat ja Zeit....").

Prüfe das Formular mit Javascript, ob es korrekt ausgefüllt wurde und ob der Standardtext angepasst wurde. Nach der erfolgreichen Prüfung soll aus dem Formular eine

E-Mail mit dem Standard-E-Mail-Programm geöffnet werden, als Betreff

verwendest Du "hallihallo" und als Empfängeradresse "mail@example.com". "*"

